Computer Organization 2026 Programming Assessment II

Mel Spectrogram on RISC-V Vector Extension

> **Announce date:** April 10, 2026
> **Exam date:** April 23 & 24, 2026

Overview

In this assessment, you will implement the core computational kernels of a **mel spectrogram** program in C, targeting the RISC-V architecture. You are expected to use RISC-V Vector (RVV) intrinsics to optimize performance.

Your developed programs are compiled by the RISC-V compiler used in the previous PA. Similar to PA 1, the compiled programs are then run on the RISC-V simulator Spike running on Ubuntu Linux 24.04.1 for performance assessment. Since PA2 is built upon open-source projects, unexpected compatibility issues may arise if you choose a different development environment.

This PA consists of three problems, with a total of 140 points. Scoring in this PA is based on correctness and performance improvement over a scalar baseline, with 60 correctness points and 80 performance points.

Suggested Workflow

1. Read this document carefully to understand the requirements of this PA.
2. Complete the implementation without RISC-V Vector.
3. Test your code using `make judge` to examine the correctness and performance of your developed programs.
4. Apply vector intrinsics to optimize the performance of your code, and test again with `make judge`. You can refer to the RISC-V V Intrinsics Specification for details on available intrinsics and their usage.
5. On the examination date in the computer classroom, you should follow the instructions listed in the **Programming Assessment Guidelines** section, and the rules in **Submission of your code** to submit your developed code.

Environment Setup

Before you start, download the ZIP file for this PA.

We recommend using the provided Docker image, which is the same environment used during the exam. It includes all required toolchains, `spike`, `pk`, and `uv` pre-installed.

If you prefer to build from scratch using your HW 0 toolchain, you must additionally install `uv` yourself, as `make judge` depends on it.

Docker

> A Docker environment will be provided during the examination in the computer classroom. Make sure you are familiar with basic Docker operations.

During the exam, the container will already be started and mounted at `~/Desktop/workspace`. Unzip the PA2 archive to `~/Desktop/workspace`; you should see `Makefile` at `~/Desktop/workspace/Makefile` and `main.c` at `~/Desktop/workspace/src/main.c`.

You can replace `...` with commands available in the container, such as `make` and `riscv64-unknown-linux-gnu-gcc`.

```
# check if container `pa2` is running.
docker ps

# start container pa2
docker start pa2

# execute commands in container
docker exec -it pa2 ...
```

PA2 uses a different image than PA1: `docker.io/asrlab/comp-org:pa2`. Outside of the exam environment, you need to pull and start the container yourself. You have a few options:

```
# mounts your current directory and removes the container on exit
docker run -it --rm -v $(pwd):/workspace docker.io/asrlab/comp-org:pa2 ...
```

```
# creates a named container you can reuse

# 1. Create and start the container (run once)
docker run -itd --name pa2 -v $(pwd):/workspace docker.io/asrlab/comp-org:pa2

# 2. Execute commands inside the container
docker exec -it pa2 ...

# 3. Open an interactive shell
docker exec -it pa2 bash

# 4. Stop the container when done
docker stop pa2
```

Makefile Commands

In most cases, you only need `make judge`, which compiles and runs your code automatically. You do not need to write full compiler and simulator commands yourself.

```
make compile     # Compile student implementation
make run         # Compile + run on Spike, write output/results.csv
make judge       # Compile + run + score results
make clean       # Remove build/ and output/
```

Compiling Your Code

Your C program is compiled using `riscv64-unknown-linux-gnu-gcc` with flags `-O3 -fno-tree-vectorize -static -march=rv64gcv -Wall`.

The `-fno-tree-vectorize` flag disables auto-vectorization, widening the gap between scalar code and manually vectorized code.

Running Your Code with Spike

The Spike RISC-V ISA simulator runs your executable with the `--isa=RV64GCV_Zicntr` flag.

```
spike --isa=RV64GCV_Zicntr /opt/riscv/riscv64-unknown-linux-gnu/bin/pk build/
bench
```

Part	Meaning
V	Vector extension (RVV), used for vectorization
Zicntr	Basic counters: `cycle`, `time`, `instret`

Performance is measured in CPU cycles read via `csrr %0, cycle` (provided by Zicntr).

Problems

Introduction to a mel spectrogram program

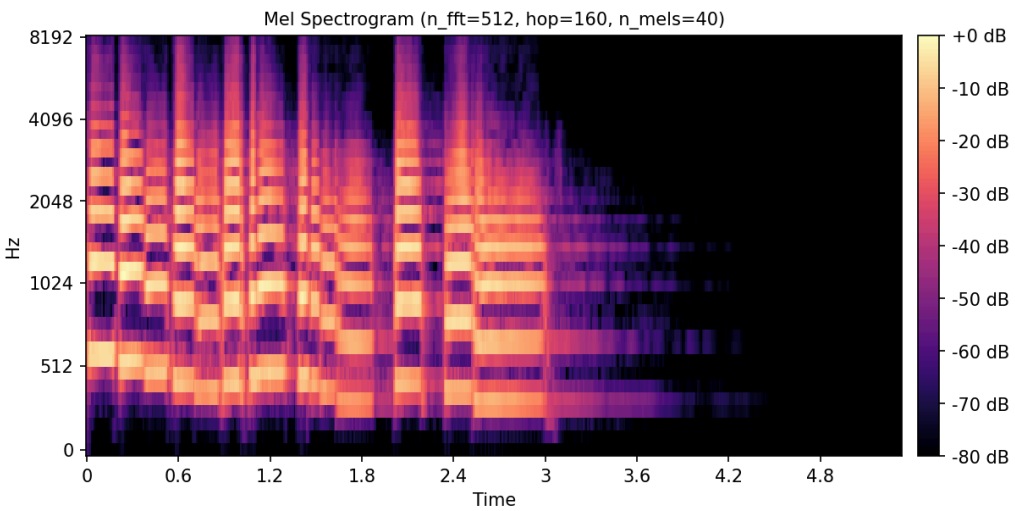

Figure 1: Mel Spectrogram

A mel spectrogram is a common audio preprocessing step in speech recognition pipelines. It converts raw audio waveforms (stored in a wave file format) into a compact representation of frequencies, grouped in the way human ears perceive pitch. The computation pipeline for a mel spectrogram relies heavily on element-wise computations, matrix multiplications, and Fourier transforms. Because these operations apply the same mathematical calculations across large arrays of data, they are inherently data-parallel and perfect candidates for RVV acceleration.

Turning audio into a mel spectrogram involves several steps. Audio loading and windowing functions are already provided for you. Your job is to focus strictly on vectorizing the heavy mathematical lifting.

The following pseudocode outlines the conceptual flow of the mel spectrogram computation.

```
void melspectrogram(audio, mel_bank, output) {
    // provided: precompute window coefficients
    hann_window(hann_buf, N_FFT);

    // provided: stft() — slide across audio frame by frame
    for (frame = 0; frame < N_FRAMES; frame++) {
        // Apply Hann window to the current audio frame (zero-padded at
boundaries).
        for (i = 0; i < N_FFT; i++) {
            fft_real[i] = audio[frame * HOP_LENGTH + i] * hann_buf[i];
            fft_imag[i] = 0;
        }
        // Problem 1: your fft() is called here
        fft(fft_real, fft_imag, N_FFT);

        // Store only the first N_FREQ_BINS = N_FFT/2+1 bins (interleaved re,
im).
        for (bin = 0; bin < N_FREQ_BINS; bin++) {
            stft_buf[frame][bin*2]   = fft_real[bin];
            stft_buf[frame][bin*2+1] = fft_imag[bin];
        }
    }

    // Problem 2: for each complex bin, compute re² + im².
    // power_buf: (N_FRAMES, N_FREQ_BINS)
    power_spectrum(stft_buf, num_frames, power_buf);

    // Problem 3: multiply the power vector of each frame by the mel filter
matrix.
    // output: (N_FRAMES, N_MELS)
    mel_filter_bank(power_buf, mel_bank, num_frames, N_MELS, N_FREQ_BINS,
output);
}
```

Problems in the mel spectrogram program

In this assessment, you will implement three functions using **RISC-V Vector (RVV) intrinsics** in `src/main.c`. Your code is compiled for the `RV64GCV` ISA and run on Spike with `Zicntr` enabled for cycle counting.

Moreover, you will be responsible for using the RVV intrinsic library to boost the performance of the program. This is achieved by writing the optimized compute kernels for the three main computational steps of the pipeline.

You will implement exactly **three functions** in `src/main.c`, which are briefly introduced below.

- **`fft`**: Compute the discrete Fourier transform of a complex signal in-place. Any correct algorithm is accepted.
- **`power_spectrum`**: Compute the power at each frequency bin from the processed `stft` data.

- `mel_filter_bank`: Apply the mel filter bank to the power spectrum.

The following constants are defined in `include/mel_spectrogram.h`:

Constant	Value	Description
`N_FFT`	512	FFT frame size
`HOP_LENGTH`	160	Samples between consecutive frames
`N_FREQ_BINS`	`N_FFT / 2 + 1`	Frequency bins per frame (= 257)
`N_MELS`	40	Number of mel bands
`MAX_FFT_N`	4096	Maximum FFT length

fft

The `fft` function takes `n` complex numbers as its input. Each complex number consists of a *real* part and an *imaginary* part. The formal parameters for the `fft` function are listed below.

Parameter	Direction	Shape	Description
`real`	input, output	(n,)	Real part of each complex sample
`imag`	input, output	(n,)	Imaginary part (may be non-zero)
`n`	input	—	Number of complex samples, power of 2, `n` ≤ `MAX_FFT_N`

> **Hint:** The Cooley-Tukey radix-2 algorithm works in two phases: first reorder inputs by bit-reversing their indices, then iteratively combine pairs of elements ("butterfly" operations) across $\log_2 n$ stages. Each stage is data-parallel; multiple butterflies can be computed simultaneously using vector registers.

power_spectrum

The `power_spectrum` function takes the output of the `stft` function as its input, where `stft` calls the `fft` function for further processing. In particular, the `power_spectrum` function computes the power at each frequency bin from the interleaved complex STFT output: $\mathrm{power}[f][k] = \mathrm{re}_{f,k}^2 + \mathrm{im}_{f,k}^2$. The concept/usage of frequency bins is illustrated in the above pseudocode (`stft_buf`). The following table lists the formal parameters for the `power_spectrum` function.

Parameter	Direction	Shape	Description
`stft_data`	input	(num_frames, N_FREQ_BINS, 2)	Interleaved `[re₀, im₀, re₁, im₁, …]` per frame
`num_frames`	input	—	Number of STFT frames (≥0)
`output`	output	(num_frames, N_FREQ_BINS)	Power value per bin

> **Hint:** Because the complex data is interleaved in memory (Real, Imag, Real, Imag…), strided memory loads (`vlse32.v`) will allow you to grab all the real parts into one vector register and all the imaginary parts into another in a single pass.

mel_filter_bank

The `mel_filter_bank` function applies the mel filter bank to the power spectrum produced by the `power_spectrum` function. This function performs the operations: $\text{output}[f][m] = \sum_{k=0}^{\text{N_FREQ_BINS}-1} \text{power}[f][k] \times \text{mel_bank}[m][k]$, where each output element is a dot product of one row of `mel_bank` against the power vector for that frame. The following table lists the formal parameters for the `mel_filter_bank` function.

Parameter	Direction	Shape	Description
`power`	input	(num_frames, N_FREQ_BINS)	Power spectrum, row-major
`mel_bank`	input	(n_mels, n_freq_bins)	Mel filter matrix, row-major
`num_frames`	input	—	Number of frames (≥0)
`n_mels`	input	—	Number of mel bands (rows of `mel_bank`; ≥0)
`n_freq_bins`	input	—	Number of frequency bins (cols of `mel_bank`; ≥0)
`output`	output	(num_frames, n_mels)	Mel-filtered result

> **Hint:** Consider how vector multiplication paired with unordered reduction sum operations (`vfredusum.vs`) can significantly accelerate these dot products.

File Structure

Important files are introduced below.

File	Edit?	Description
`src/main.c`	✓	Student implementation (only submitted file)
`data/`	✗	Judge input/expected files
`include/mel_spectrogram.h`	✗	Constants and function signatures
`scripts/judge.py`	✗	Score calculation used by `make judge`
`scripts/mel_spectrogram.py`	✗	Simplified exam ground-truth reference
`scripts/reference.py`	✗	Full librosa reference pipeline for cross-checking
`src/bench.c`	✗	Benchmark/judge harness
`src/utils.c`	✗	Provided pipeline (`hann_window`, `stft`, `melspectrogram`)
`Makefile`	✗	Build and judge commands

Scoring

PA2 has a total of **140 points**, 60 points for correctness and 80 points for performance. Performance points are awarded only if the corresponding correctness test passes.

> The judge uses different random seeds than the local `data/` files. The file format and test case structure are identical, but the numeric values differ.

Correctness (60 pts)

Correctness is verified against ground-truth data generated from `scripts/mel_spectrogram.py`.

`scripts/reference.py` and `librosa.feature.melspectrogram` are provided as reference validation resources.

Floating-point results are not expected to be bit-exact as our ground truth. Your output is considered correct if the output is bounded by the range `diff`, where `diff ≤ 1e-5 + 1e-4 * |ground truth|`.

Group	Cases	Points/case	Subtotal
fft	5	6	30
power_spectrum	5	3	15
mel_filter_bank	5	3	15
Total	**15**		**60**

Test files for `fft`: `fft_4`, `fft_8`, `fft_64`, `fft_512`, `fft_4096`.

Test files for `power_spectrum`: `power_zeros`, `power_1frame`, `power_small`, `power_medium`, `power_large`.

Test files for `mel_filter_bank`: `mel_tiny`, `mel_small`, `mel_medium`, `mel_dense`, `mel_large`.

Performance (80 pts)

Each benchmark is worth **20 points**, awarded in four tiers of 5 points each. Your speedup is calculated by `baseline_cycles / your_cycles`. If your cycle count exceeds the baseline (speedup < 1.0×) you receive **0 points** for that benchmark, even if correctness passes.

Test	Baseline	0 pts	5 pts	10 pts	15 pts	20 pts
fft	1,590,000	< 1.0×	≥ 1.0×	≥ 1.1×	≥ 1.25×	≥ 1.45×
power_spectrum	113,000	< 1.0×	≥ 1.0×	≥ 2.5×	≥ 5.0×	≥ 10.0×
mel_filter_bank	3,130,000	< 1.0×	≥ 1.0×	≥ 2.0×	≥ 4.0×	≥ 8.0×
melspectrogram	24,600,000	< 1.0×	≥ 1.0×	≥ 1.2×	≥ 1.45×	≥ 1.75×

For example, `≥ 1.45×` in the fft row for 20 pts means your cycle count must be at most `1,590,000 / 1.45 ≈ 1,096,552`.

Your code will be compiled with auto-vectorization disabled, ensuring that any performance gains come from your explicit use of RVV intrinsics. The compiler flag is shown in the **Compiling Your Code** section.

Local Judge

To get the score of your implementation, run `make judge` in your environment; the outputs will look like this. This local judge performs the tests for the three functions at once with different test cases.

We do not provide a test for a single function. For example, if you want to test your implementation of `fft`, you should use `make judge`.

In the `Correctness` section, the output shows whether your implementation passes each test.

In the `Performance` section, *Cycles* is the measured performance of your implementation. The *Cycles* value is used to calculate your earned points based on the achieved speedup, as defined in the **Scoring** section.

```
        Correctness (60 pts)
```

Test	Result	Points
fft_4	PASS	6/6
fft_8	PASS	6/6
fft_64	PASS	6/6
fft_512	PASS	6/6
fft_4096	PASS	6/6
power_zeros	PASS	3/3
power_1frame	PASS	3/3
power_small	PASS	3/3
power_medium	PASS	3/3
power_large	PASS	3/3
mel_tiny	PASS	3/3
mel_small	PASS	3/3
mel_medium	PASS	3/3
mel_dense	PASS	3/3
mel_large	PASS	3/3
Subtotal		60/60

```
          Performance (80 pts)
```

Test	Cycles	Speedup	Points
fft_4096	1,580,216	1.00x	5/20
power_large	112,432	1.00x	5/20
mel_large	3,100,421	1.00x	5/20
melspec_trumpet	24,459,131	1.00x	5/20
Subtotal			20/80

```
──────── Final Score ────────
Correctness:    60 / 60
Performance:    20 / 80
Total:          80 / 140
```

Programming Assessment Guidelines

Rules

- Please bring your student ID card as proof of identity.
- During the exam, only https://moodle.ncku.edu.tw/ is accessible, and you may use it only to download the exam and upload your submission.
- The use of USB/external devices, cheat sheets, paper notes, or any AI tools is strictly prohibited. **Violations of the above rules will be regarded as cheating, and the exam score will be recorded as 0.**
- There will be no hints provided during the exam.

Submission of your code

You are only allowed to modify `src/main.c`. You are not allowed to add any files, for example, adding an included C file or modifying the Makefile.

Make sure to disable or remove any debugging prints before submission, as they can heavily skew cycle counts and performance metrics.

Upload `src/main.c` to NCKU Moodle. You do not need to rename the file; submit a single `main.c` without any nested folders. Again, `src/main.c` is the only file that will be uploaded and handled.

> - Uploading an unexpected filename that causes the judge to be unable to score it automatically will result in a loss of 10 points.
> - Unexpected file content that prevents the judge from continuing will result in 0 points.
> - Compiling or linking errors that prevent the judge from continuing will result in 0 points.

RISC-V Vector Extension Primer

The RISC-V V extension uses a **variable-length vector model**. The application sets the active vector length with `vsetvl`, then operates on up to `vl` elements per instruction. At VLEN=128 with 32-bit floats and LMUL=1, each register holds 4 elements; LMUL=2 gives 8, LMUL=4 gives 16.

The canonical vectorized loop structure is:

```
#include <riscv_vector.h>

for (size_t i = 0; i < n; ) {
    size_t vl = __riscv_vsetvl_e32m4(n - i);
    // ... load, compute, store using vl elements ...
    i += vl;
}
```

Key intrinsics for assessment:

Intrinsic pattern	Operation
`__riscv_vsetvl_e32mN`	Set vector length for f32, LMUL=N
`__riscv_vle32_v_f32mN`	Unit-stride load
`__riscv_vlse32_v_f32mN`	Strided load (useful for interleaved data)
`__riscv_vse32_v_f32mN`	Unit-stride store
`__riscv_vfmul_vv_f32mN`	Element-wise multiply
`__riscv_vfmacc_vv_f32mN`	Fused multiply-accumulate (`acc += a * b`)
`__riscv_vfnmsac_vv_f32mN`	Fused negate-multiply-subtract
`__riscv_vfredusum_vs_f32mN_f32m1`	Unordered floating-point reduction sum
`__riscv_vfmv_v_f_f32m1`	Splat scalar into vector
`__riscv_vfmv_f_s_f32m1_f32`	Extract scalar from vector element 0

Include `<riscv_vector.h>` to access these intrinsics. For a complete reference and usage, see the RISC-V V Intrinsics Specification or the intrinsic documentation repository.

Reference

Offline versions of these references will be provided during the exam.

- RISC-V V Intrinsics Specification (PDF): complete RVV intrinsic reference
- RISC-V V Intrinsics Documentation (GitHub): intrinsic documentation source
- RISC-V ISA Manual (PDF): unprivileged ISA specification
- RISC-V ISA Manual (GitHub): ISA manual source
- librosa.feature.melspectrogram: reference implementation
- Cooley-Tukey FFT algorithm: the recommended FFT implementation
- Getting to Know the Mel Spectrogram: introductory overview of mel spectrograms